

DAT22-3

Software Engineering & Code Design

Terminal, Git, environments, OOP design, and testing

Tools that make data work reproducible and code that stays maintainable.

Albert School — 28 hours

Contents

1	The terminal	2
1.1	Navigation	2
1.2	File and directory operations	2
1.3	Text manipulation	3
1.4	Bash scripting	3
2	Version control with Git	3
2.1	Local repositories and commits	3
2.2	Connecting to GitHub	4
2.3	Branching and merging	4
2.4	Pull requests and code review	4
2.5	Resolving merge conflicts	4
3	Reproducible environments	4
4	Collaborative project structure	5
5	Object-oriented design	5
5.1	Inheritance	5
5.2	Composition	5
5.3	Choosing between them	6
6	Encapsulation and dunder methods	6
7	Refactoring to object-oriented design	7
8	Unit testing with pytest	7

1 The terminal

The terminal runs a shell that executes typed commands. Commands below are for Bash.

1.1 Navigation

- `pwd` — print the working directory.
- `ls` — list the contents of the working directory; `ls -l` adds details, `ls -a` shows hidden files.
- `cd path` — change the working directory; `cd ..` moves up one level, `cd ~` goes to the home directory.
- `find path -name "pattern"` — search for files matching a name pattern under a directory.

```
pwd
ls -la
cd projects/data
find . -name "*.csv"
```

1.2 File and directory operations

- `mkdir name` — create a directory.
- `touch name` — create an empty file.
- `cp src dst` — copy; `cp -r` copies a directory.
- `mv src dst` — move or rename.
- `rm name` — delete a file; `rm -r` deletes a directory.

```
mkdir data
touch data/raw.csv
cp data/raw.csv data/backup.csv
mv data/backup.csv data/old.csv
rm data/old.csv
```

1.3 Text manipulation

- `cat file` — print the whole file.
- `head file` — print the first lines; `head -n 20` sets the count.
- `tail file` — print the last lines; `tail -n 20` sets the count.
- `grep "pattern" file` — print lines matching a pattern; `grep -i` ignores case, `grep -r` searches recursively.

Redirection sends output to or from files, and pipes pass output between commands:

- `command > file` — write output to a file, overwriting it.
- `command >> file` — append output to a file.
- `command < file` — read input from a file.
- `command1 | command2` — pass the output of one command as input to the next.

```
grep "error" log.txt > errors.txt
cat data.csv | head -n 5
grep -c "error" log.txt >> counts.txt
```

1.4 Bash scripting

A Bash script is a file of commands run in sequence. It begins with a shebang line naming the interpreter, and is made executable with `chmod +x`.

```
#!/bin/bash
for file in *.csv; do
    echo "Lines in $file:"
    wc -l "$file"
done
chmod +x count.sh
./count.sh
```

Variables are assigned without spaces (`name=value`) and read with `$name`. Scripts automate repetitive file operations such as renaming, counting, or filtering many files at once.

2 Version control with Git

Version control records the history of a project as a sequence of snapshots, allowing changes to be tracked, reverted, and combined across collaborators. Git is a version control system; GitHub is a hosting service for Git repositories. It is essential for reproducible, collaborative work because every change is attributed, recoverable, and reviewable.

2.1 Local repositories and commits

A repository tracks a project directory. Changes move through three stages: the working directory, the staging area, and the commit history.

```
git init                # create a repository
git status              # show the state of files
git add file.py         # stage a change
git add .               # stage all changes
git commit -m "message" # record staged changes as a commit
git log                 # show the commit history
```

A meaningful commit message states what the commit changes and why, in the imperative mood, for example `Add CSV loader for sales data`.

2.2 Connecting to GitHub

```
git remote add origin https://github.com/user/repo.git
git push -u origin main # upload commits to GitHub
git pull                # download and merge remote commits
git clone https://github.com/user/repo.git
```

2.3 Branching and merging

A branch is an independent line of work. Developing a feature on its own branch keeps the main branch working.

```
git branch feature      # create a branch
git checkout feature    # switch to it
git checkout -b feature # create and switch in one step
git merge feature       # merge a branch into the current one
```

The workflow is: create a branch, commit work on it in isolation, then merge it back into main.

2.4 Pull requests and code review

A pull request (PR) proposes merging one branch into another on GitHub and opens the change for review. The workflow is:

1. Push the feature branch to GitHub.
2. Open a pull request from the feature branch into main.
3. A reviewer leaves structured comments on specific lines, requesting changes or approving.
4. The author addresses the feedback with new commits.
5. Once approved, the pull request is merged.

A structured review comment identifies the location, states the issue, and proposes a concrete change.

2.5 Resolving merge conflicts

A merge conflict occurs when two branches change the same lines. Git marks the conflict in the file:

```
<<<<<< HEAD
total = price * 1.2
=====
total = price * 1.20 + shipping
>>>>>> feature
```

The lines between <<<<<< and ===== come from the current branch; those between ===== and >>>>>> come from the merged branch. Resolve the conflict by editing the file to the correct final version, removing the three marker lines, then staging and committing:

```
git add file.py
git commit
```

3 Reproducible environments

A virtual environment is an isolated Python installation for a single project. It keeps each project's dependencies separate, so that different projects can use different package versions without interfering.

```
python -m venv venv      # create an environment named venv
source venv/bin/activate # activate it (Linux/macOS)
```

```
venv\Scripts\activate    # activate it (Windows)
deactivate                # leave the environment
```

Packages are installed with `pip` while the environment is active. The list of dependencies is recorded in a `requirements.txt` file:

```
pip install pandas
pip freeze > requirements.txt    # record installed packages
pip install -r requirements.txt  # reproduce on another machine
```

To reproduce a setup on a different machine: clone the repository, create and activate a new virtual environment, then install from `requirements.txt`. Pinning versions (`pandas==2.2.0`) in `requirements.txt` makes the reproduction exact.

4 Collaborative project structure

A data project repository uses standard directories so that collaborators find files where they expect them:

- `data/` — input datasets (often excluded from Git for size or privacy).
- `src/` — source code.
- `notebooks/` — exploratory Jupyter notebooks.
- `outputs/` — generated results: figures, reports, processed files.

A `README.md` file is the project contract: it states what the project does, how to set it up (environment and dependencies), and how to run it.

Work is divided across branches so that several people contribute without breaking `main`: each task is developed on its own branch and merged back through a reviewed pull request.

5 Object-oriented design

A class defines a type of object by its attributes (data) and methods (behaviour). Object-oriented design models a data project as a set of collaborating classes, such as a `DataLoader`, a `Scraper`, or a `Pipeline`.

5.1 Inheritance

Inheritance defines a class as a specialized version of another, reusing and extending its behaviour. It expresses an **is-a** relationship.

```
class DataLoader:
    def load(self):
        raise NotImplementedError

class CSVLoader(DataLoader):    # a CSVLoader is a DataLoader
    def load(self):
        ...

class JSONLoader(DataLoader):   # a JSONLoader is a DataLoader
    def load(self):
        ...
```

5.2 Composition

Composition builds a class by holding instances of other classes as attributes. It expresses a **has-a** relationship.

```

class Pipeline:
    # a Pipeline has a loader and a cleaner
    def __init__(self, loader, cleaner):
        self.loader = loader
        self.cleaner = cleaner

    def run(self):
        data = self.loader.load()
        return self.cleaner.clean(data)

```

5.3 Choosing between them

Use inheritance when one class is a specialized kind of another and shares its interface. Use composition when a class is built from parts that each have their own responsibility. Composition is preferred when the relationship is **has-a** rather than **is-a**, as it keeps classes loosely coupled.

6 Encapsulation and dunder methods

Encapsulation hides the internal state of an object behind a public interface. Attributes intended to be private are prefixed with an underscore by convention; methods exposed to users form the object's public contract.

```

class Account:
    def __init__(self, balance):
        self._balance = balance        # private by convention

    def deposit(self, amount):
        self._balance += amount        # public method

    def get_balance(self):
        return self._balance           # public contract

```

Dunder (double-underscore) methods let custom objects behave like Python built-ins:

- `__str__` — the readable string shown by `print` and `str`.
- `__len__` — the value returned by `len`.
- `__eq__` — the comparison used by `==`.

```

class Dataset:
    def __init__(self, rows):
        self._rows = rows

    def __str__(self):
        return f"Dataset with {len(self._rows)} rows"

    def __len__(self):
        return len(self._rows)

    def __eq__(self, other):
        return self._rows == other._rows

d = Dataset([1, 2, 3])
print(d)          # Dataset with 3 rows
len(d)            # 3
d == Dataset([1, 2, 3])  # True

```

7 Refactoring to object-oriented design

Refactoring rewrites code to improve its structure without changing its behaviour. A procedural data script — a sequence of functions passing data through global variables — is refactored into classes that group related data and behaviour.

A procedural script:

```
def load(path):
    ...

def clean(data):
    ...

def summarize(data):
    ...

data = load("sales.csv")
data = clean(data)
report = summarize(data)
```

becomes an object-oriented design:

```
class SalesReport:
    def __init__(self, path):
        self.path = path
        self.data = None

    def load(self):
        ...

    def clean(self):
        ...

    def summarize(self):
        ...

report = SalesReport("sales.csv")
report.load()
report.clean()
summary = report.summarize()
```

Each modeling decision is justified: what becomes a class (a coherent entity with its own data and behaviour), what becomes a method (an operation on that data), and what becomes an attribute (state shared across the operations).

8 Unit testing with pytest

A unit test checks that a small piece of code produces the expected result. Automated tests prevent regressions: on a collaborative project, they catch changes that break existing behaviour before they reach main.

pytest discovers functions named `test_*` and runs them. A test uses `assert` statements; the test passes if every assertion holds.

```
# test_account.py
from account import Account

def test_deposit():
```

```
a = Account(100)
a.deposit(50)
assert a.get_balance() == 150
```

Run the tests from the terminal:

`pytest`

A fixture provides reusable setup shared by several tests. It is defined with the `@pytest.fixture` decorator and requested by naming it as a parameter:

```
import pytest
from account import Account

@pytest.fixture
def account():
    return Account(100)

def test_deposit(account):
    account.deposit(50)
    assert account.get_balance() == 150

def test_initial_balance(account):
    assert account.get_balance() == 100
```

A test case names the behaviour under test, sets up the inputs, calls the code, and asserts the expected result.

Exercises

1. From the terminal, create a directory, move into it, create a file, then use `find` to locate it.
2. Use `grep` with redirection to extract all lines containing a word from one file into a new file.
3. Write a Bash script that loops over every `.csv` file in a directory and prints its line count.
4. Initialize a Git repository, make two commits with meaningful messages, and display the history with `git log`.
5. Create a branch, commit a change on it, and merge it back into `main`.
6. Open a pull request on GitHub, leave a structured review comment, address it with a new commit, and merge.
7. Create a merge conflict deliberately, then resolve it manually and complete the merge.
8. Create a virtual environment, install a package, record it in `requirements.txt`, and reproduce the environment from that file.
9. Set up a repository with `data/`, `src/`, `notebooks/`, and `outputs/` directories and a README stating how to run the project.
10. Design a small class hierarchy using inheritance, and a class using composition, and explain which relationship each models.
11. Write a class with a private attribute, public methods, and the `__str__`, `__len__`, and `__eq__` dunder methods.
12. Refactor a procedural data script into an object-oriented design and justify each modeling decision.
13. Write pytest tests for a class, including at least one fixture, and run them from the terminal.